基于eBPF的移动终端持续剖析与即时诊断实践

The 4th eBPF Developer Conference (eBPFDC 2026)

小米科技有限责任公司 | 内核技术部

张威 / 夏鸿 / 王振淦

陕西·西安　　2026年4月18日

目录

作者介绍

背景介绍

方案设计

演进思考

参考资料

作者介绍

张威　eBPF技术爱好者，Android系统性能优化工程师，有多年Android系统性能优化经验，乐于通过eBPF探索和构建可观测能力。

夏鸿　eBPF技术爱好者，性能工程专家，致力于让性能分析与优化变得更简单。在系统性能可观测、性能剖析与自动化诊断等领域有多年的实战经验。

王振淦　eBPF技术爱好者，性能DFP领域专家，有多年性能优化和维测经验，致力于构建端侧可观测能力体系。

背景介绍

移动终端的性能故障具有强实时性。

用户感知到卡顿的瞬间，现场上下文数据量庞大且稍纵即逝，难以持续采集。而观测手段本身若过重，反而会加剧系统负载、放大卡顿，形成"观测者效应"。

背景介绍

方案	原理	开销	实时性	闭环能力	适用场景
Systrace / Perfetto	用户态埋点 + ftrace	中（3~8%）	离线	X	开发调试
Simpleperf	采样式 CPU Profiler	中高（5~10%）	离线	X	性能分析
APM SDK / Firebase	用户态 Hook	高（5~10%）	准实时	部分	线上调试
eBPF即时诊断	内核态探针	<1%	实时	✓	线上治理

背景介绍

我们基于eBPF构建了持续剖析与即时诊断系统：内核态部署轻量常驻探针，对资源关键路径持续低开销观测；一旦感知到丢帧、卡顿等体验恶化，动态激活精细化eBPF诊断插件进行深度采集，实现故障现场实时捕获与即时归因。

与以往“先采集、后分析”模式不同，这套方案具备低开销、响应快、故障可自愈三大核心能力，形成从感知到处置的全程在线闭环，为移动终端性能实时治理提供了新思路。

用户态信息采集

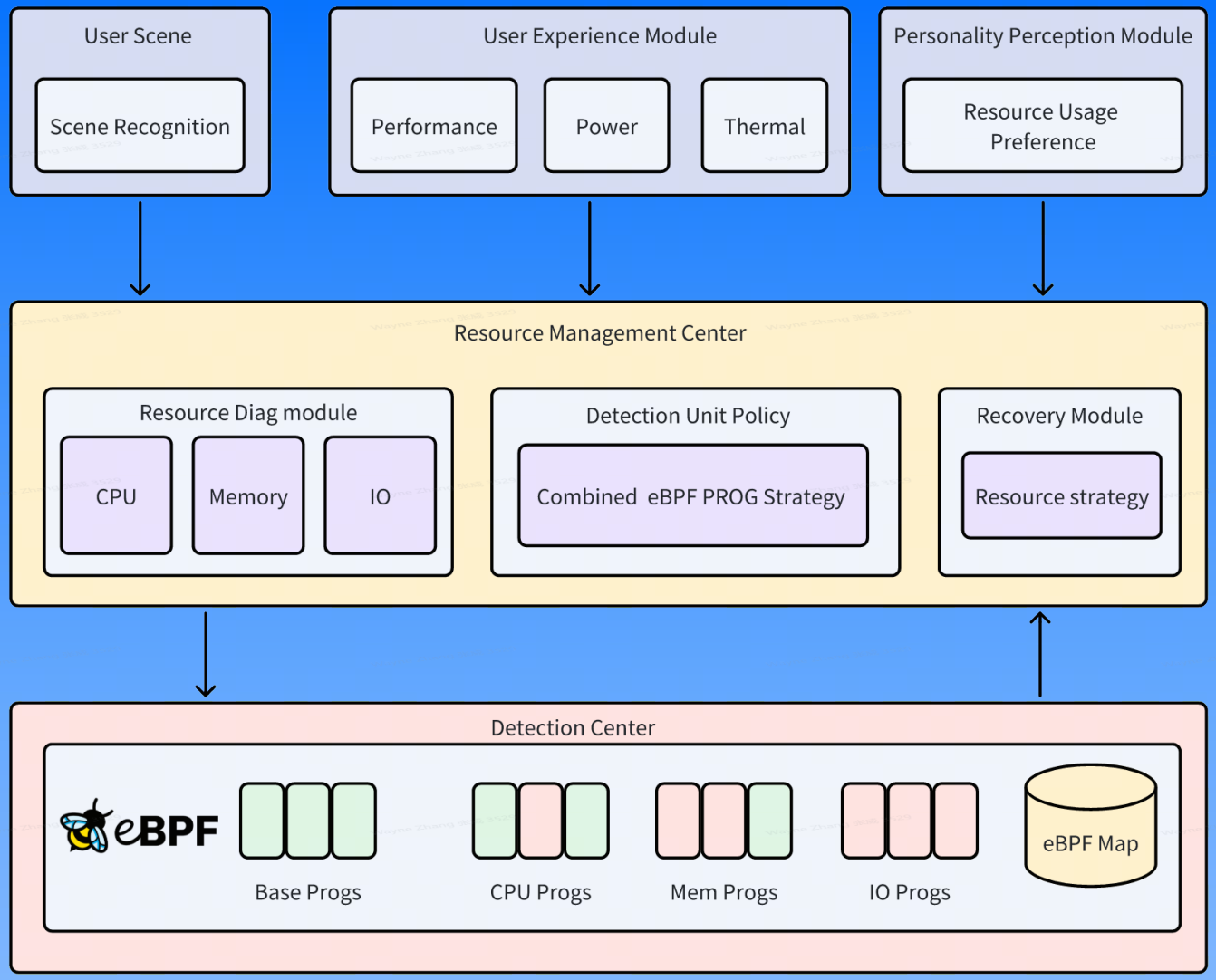
- 用户使用场景
- 用户体验状态（性能、功耗等）
- 用户资源画像

资源管理中心

- 资源供给诊断模块
- eBPF Prog 组合策略
- 资源优化策略下发

内核态信息采集（eBPF）

- 线程资源等待状态
- 资源抢占Owner



基于eBFP的资源分析诊断方案

感知学习
- 执行路径记录
- 系统资源快照
- 事件时序日志
- 异常特征分类
- 问题场景分类

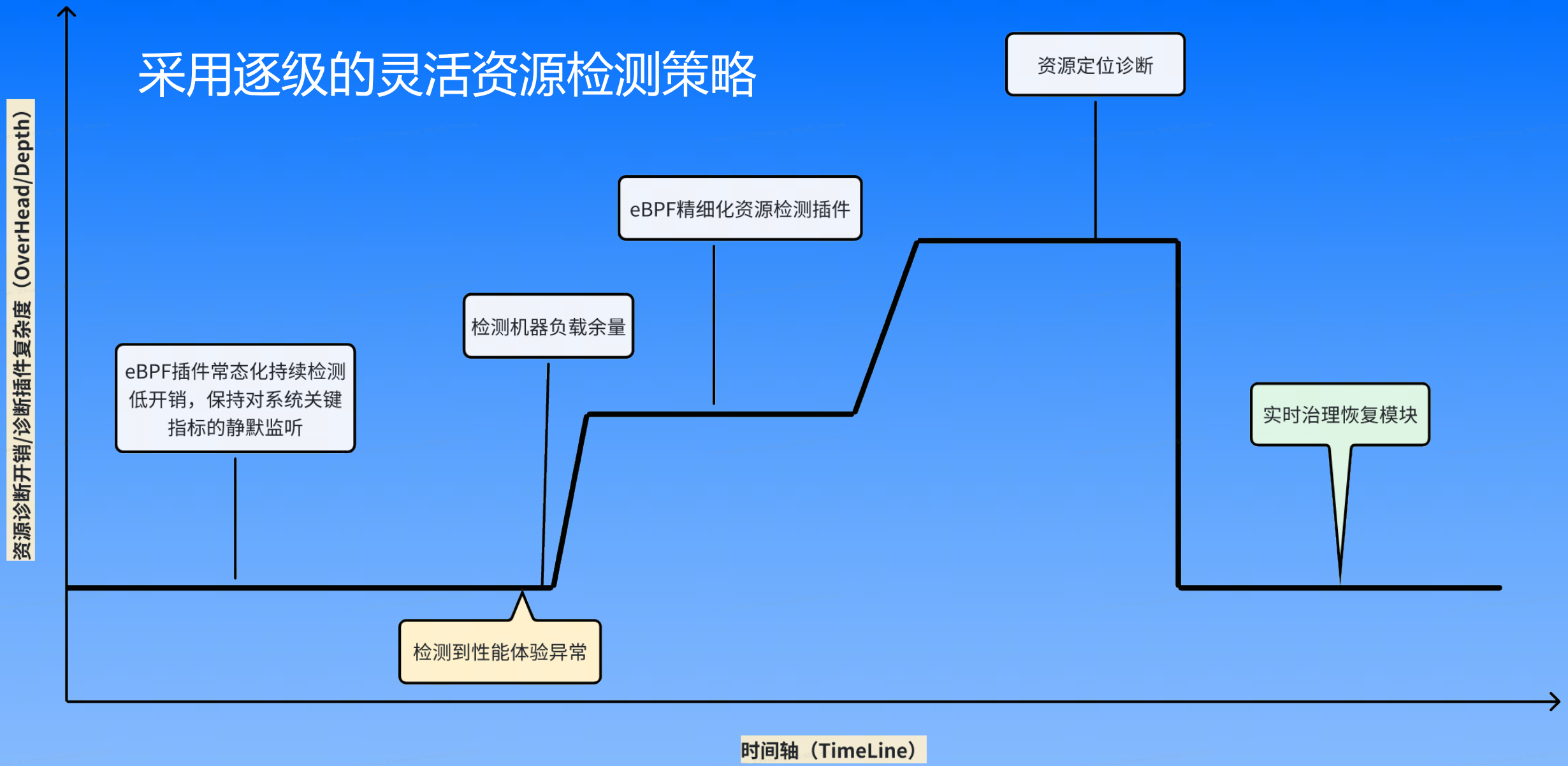
分析建模
- 超时检测模型
- 时延分布模型
- 依赖关系模型
- 资源供给模型
- 负载压力模型

归因分析
- 资源供给分析
- 因果链路分析
- 异常占用溯源
- 基线偏差分析
- 多维结合定界

故障处置
- 进程冻结管控
- 进程精准查杀
- 资源分层管理
- 优先级调整
- 内存压缩回收

诊断调试
- 故障受控复现
- 版本对比分析
- 故障回放验证
- 准确性评估
- 自愈有效性验证

采用逐级的灵活资源检测策略

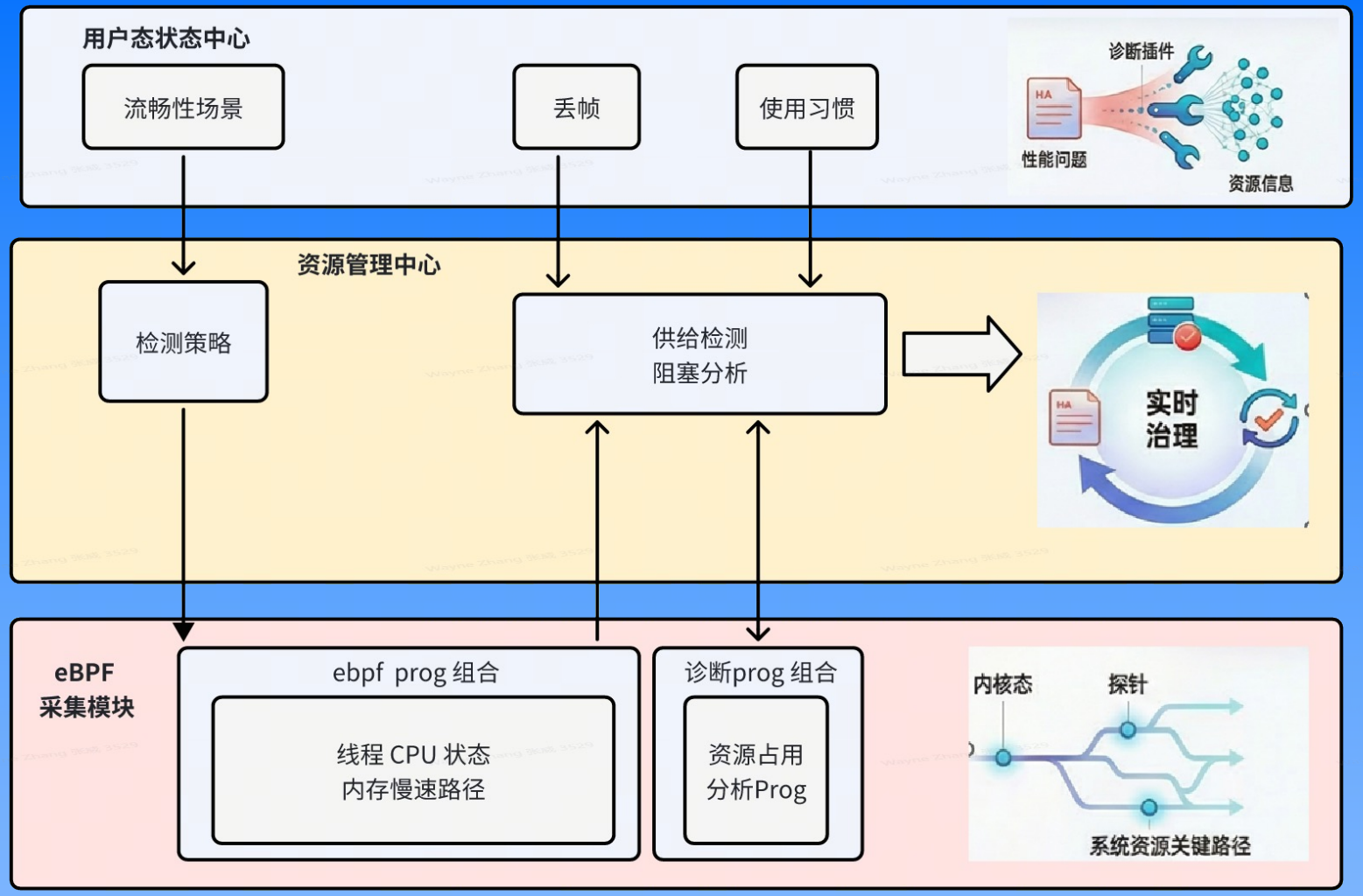


根据用户场景灵活配置eBPF检测组件

当检测到卡顿、丢帧等体验指标恶化时，开启占用分析eBPF插件，找到资源分配瓶颈进程

识别资源抢占进程

通过优先级队列、进程冻结、进程查杀等策略释放阻塞瓶颈，实时调整资源供给



资源供给诊断实践

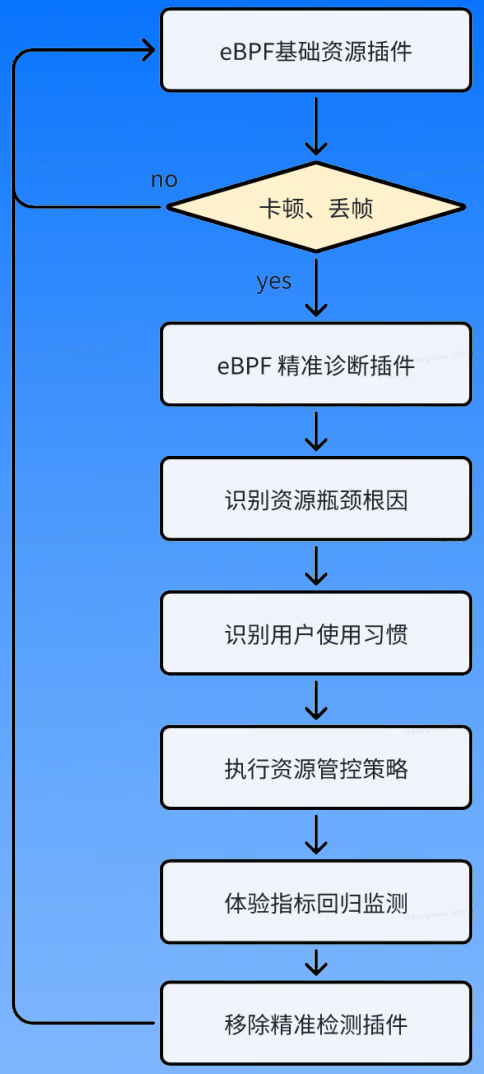
① 轻量探针常驻 - 内核态持续低开销观测
挂载 mm_vmscan_direct_reclaim_begin / end
静默监控指定线程内存压力，感知直接内存回收路径是否被触发
开销极低，常态运行不影响用户体验

② 体验恶化 · 触发精准诊断插件组合
（时间）用户态检测到丢帧/卡顿，资源管理中心按需激活精准 eBPF 插件：
· try_to_free_pages — 追踪内存回收触发路径
· remove_mapping — 定位被驱逐的页面来源进程
· mm_page_alloc — 识别触发回收的内存申请方

③ 识别资源抢占来源 · 用户画像 + 内核数据联合定界
eBPF 定位内存占用最大的非前台进程（如后台视频/新闻类应用）
结合用户习惯特征加权（近期最少使用、非常驻 Top 应用）

④ 执行优化策略 - 进程查杀 + 内存回收
对低优先级抢占进程执行查杀/压缩，释放内存瓶颈
资源管理中心同步下发调度优先级策略，保障前台应用供给

⑤ 效果验证 · 体验指标回归监测
监测丢帧率/帧渲染耗时是否恢复基线，未恢复则重新触发诊断链路



关键代码/资源检测插件

```
bpf/
├── include/
│   └── common.h                    # 全局公共类型、事件结构体、map 宏
│
├── memory/                         # 内存资源检测
│   ├── common/
│   │   └── mem_common.h            # 内存类公共结构（压力等级、精细化事件、统计 map）
│   ├── direct_reclaim.bpf.c        # 直接内存回收阻塞检测
│   ├── try_to_free_pages.bpf.c     # 内存回收触发路径追踪
│   ├── page_alloc.bpf.c            # 页面分配申请方识别
│   ├── remove_mapping.bpf.c        # 页面驱逐来源溯源
│   ├── mem_compaction.bpf.c        # 内存碎片整理 & kswapd 监控
│   ├── lmk_monitor.bpf.c           # Android LMK / OOM 查杀监控
│   ├── ion_monitor.bpf.c           # DMA-buf / Ion 图形内存分配追踪
│   ├── mem_fault.bpf.c             # 缺页异常精细化检测（minor/major/COW）
│   ├── slab_monitor.bpf.c          # Slab/kmalloc 大分配 + shrink 追踪
│   ├── zram_monitor.bpf.c          # zRAM 压缩 swap I/O 延迟监控
│   ├── memcg_monitor.bpf.c         # memcg 配额回收 / OOM 检测
│   ├── swap_monitor.bpf.c          # 通用 swap 换出/换入延迟监控
│   ├── mmap_monitor.bpf.c          # 大块匿名内存映射追踪（内存暴涨溯源）
│   ├── cma_monitor.bpf.c           # CMA 连续物理内存分配延迟（camera/codec）
│   ├── writeback_monitor.bpf.c     # 脏页写回与 dirty throttle 阻塞
│   ├── rss_monitor.bpf.c           # 进程 RSS 暴涨 / 泄漏侦测
│   ├── oom_monitor.bpf.c           # 全局 OOM killer 事件（区别于 memcg OOM）
│   ├── thp_monitor.bpf.c           # THP 大页 collapse/split 延迟追踪
│   ├── page_cache_monitor.bpf.c    # 页缓存 add/evict + readahead 追踪
│   ├── vmalloc_monitor.bpf.c       # 内核 vmalloc 大块分配追踪
│   ├── buddy_monitor.bpf.c         # 伙伴分配器碎片化 + 慢路径检测
│   └── test/
│       ├── test_direct_reclaim.c
│       ├── test_page_alloc.c
│       ├── test_mem_fault.c
│       ├── test_slab_monitor.c
│       ├── test_zram_monitor.c
│       ├── test_memcg_monitor.c
```

关键探针 / 插件	检测能力
sched_switch	CPU 抢占与调度延迟
sched_wakeup	唤醒等待延迟
cpu_frequency	CPU 调频事件
sched_stat_runtime	实际运行时长统计
rss_stat	进程内存 RSS 变化
oom_score_adj_update	OOM 优先级动态变化
ion_buffer_alloc	Ion/DMA-buf 分配追踪
mm_vmscan_direct_reclaim	回收延迟追踪
block_rq	块设备 I/O 请求全链路延迟
binder_transaction	Binder 事务端到端延迟
binder_lock / binder_unlock	Binder 锁竞争分析
…	…

故障响应收益

指标	传统方案	本方案
故障感知延迟	小时~天级	< 5min
数据完整率	~40%	> 60%
根因定位时间	人工分析 2~4h	自动诊断 < 1min
闭环处置率	需人工介入	自动治理

指标	改善情况
性能工单数量	减少 30 %（自动治理吸收）
开发者调试效率	精准堆栈 + 根因报告，无需手动复现
问题平均修复周期	14天 → 8 小时

演进思考

短期演进（~6 个月）

- 扩展诊断覆盖： eBPF 可观测能力扩展建设
- 规则库沉淀：基于线上故障学习，自动更新 eBPF Prog 组合策略
- 跨进程 IPC 延迟分析：Binder / Socket 阻塞链路端到端溯源
- 收益量化：建立故障定界准确率与闭环周期的持续度量体系

中长期演进（~12 个月）

- AI 驱动智能诊断：构建性能故障特征库（eBPF 数据标注）
- 训练轻量异常检测模型，端侧实时推理，逐步替代规则引擎
- 闭环自愈：诊断结果直接驱动调度/资源策略，实现无人值守
- 云端协同：故障模式上报云端聚合，持续优化端侧诊断规则

参考资料

1. BPF Instruction Set
https://docs.kernel.org/bpf/standardization/instruction-set.html

2. Extended Android Tools (Meta)
https://github.com/facebookexperimental/ExtendedAndroidTools

感谢聆听

关于我们：

小米科技有限责任公司
内核技术部-DFP业务团队

陕西·西安　2026年4月18日